Milestone 6: Retrieval-Augmented Generation (RAG) & Agentic Pipeline

MLOps Course - Module 7

Aligned Learning Objectives

- **CG3.LO3:** Design RAG systems integrating retrievers, vector databases, and model endpoints for knowledge-grounded responses.
- **CG3.LO4:** Build and evaluate agentic AI systems that coordinate reasoning, tool use, and multi-step task execution.

Why This Matters

Retrieval-augmented generation (RAG) and agentic AI represent the frontier of production AI systems. RAG enables models to ground their responses in external knowledge, reducing hallucinations and improving factual accuracy. Agentic systems coordinate multiple tools and reasoning steps to solve complex tasks autonomously.

In this milestone, you will build both capabilities: a complete RAG pipeline that retrieves and generates grounded responses, and an agent controller that intelligently orchestrates multiple tools including retrieval. These skills are essential for deploying knowledge-grounded AI systems in production environments where accuracy, transparency, and multi-step reasoning are critical.

This milestone directly prepares you for the final project's monitoring and governance requirements by establishing the foundational systems you'll need to observe, evaluate, and maintain.

Assignment Overview

You will construct a complete retriever-agent pipeline demonstrating both RAG capabilities and agentic coordination. This milestone integrates two core components:

1. **RAG Pipeline:** A functional retrieval-augmented generation system with proper chunking, indexing, retrieval, and grounded generation
2. **Agent Controller:** A multi-tool agent that intelligently selects between retrieval and other operations to solve multi-step tasks

The assignment emphasizes architectural understanding, evaluation rigor, and the ability to coordinate retrieval with reasoning in production-like scenarios.

Required Model Policy

This milestone requires a **real open-weight instruct LLM** for both grounded generation and agent behavior. Debugging with mocks is fine during development, but ** mock generators, template-only outputs, or extractive-only substitutes do not satisfy the submission requirements**.

Your submission must use:

- **One open-weight instruct model in the 7B-14B class** for final evaluation runs
- **Local or self-hosted inference** (local machine or cloud VM/notebook you control)
- **No proprietary hosted inference APIs** unless you receive explicit instructor approval for an accommodation

Recommended model families include:

- `mistralai/Mistral-7B-Instruct-v0.3`
- `meta-llama/Llama-3.1-8B-Instruct`
- `Qwen/Qwen2.5-7B-Instruct`
- `Qwen/Qwen2.5-14B-Instruct`

If you cannot run a 7B-14B model due to hardware or quota limits, document the constraint in your README and evaluation report and obtain instructor approval before substituting a smaller open-weight model.

Part 1: RAG Pipeline Implementation (5 points)

Requirements

You will implement a minimal but complete RAG pipeline using any framework (LangChain, LlamaIndex, DSPy, or custom implementation). Your pipeline must demonstrate the full RAG workflow from document ingestion through grounded generation with a **real open-weight 7B-14B instruct LLM**.

Deliverables

1. **Notebook or script** performing:

- Document ingestion with appropriate chunking strategy
- Embedding generation and vector index creation
- Retrieval query execution
- Grounded answer generation using a real open-weight 7B-14B instruct LLM

2. **Evaluation report** (2 pages) covering:
   - **Retrieval accuracy** on 10 handcrafted queries
     - Include metrics like precision@k, recall@k, or custom relevance scores
   - **Qualitative grounding analysis**
     - Assess how well generated responses are grounded in retrieved context
     - Identify cases of hallucination or missing citations
   - **Error attribution**
     - Distinguish retrieval failures from generation/grounding failures
     - Note any model-capacity limitations you observed
   - **Latency measurements** for:
     - Retrieval operations
     - Generation operations
     - End-to-end pipeline
3. **Pipeline diagram** (ASCII art or simple UML) showing:
   - Data flow from documents to final output
   - Key components (chunker, embedder, vector store, retriever, generator)
   - Decision points and data transformations
4. **Environment file** (`requirements.txt` or lockfile) for reproducibility
5. **Model deployment note** (in README or separate markdown file) documenting:
   - Model name and exact version
   - Parameter class (7B, 8B, 14B, etc.)
   - Serving method (Ollama, vLLM, Hugging Face Transformers, TGI, etc.)
   - Hardware/runtime environment used for final evaluation
   - Typical generation latency observed in your setup

Rubric (5 pts)

Criteria	Points	Description
Correct implementation of retriever + generator pipeline	2	Pipeline successfully performs ingestion, retrieval, and generation with a real open-weight 7B-14B instruct LLM; code is functional and well-structured
Clear evaluation of retrieval accuracy + grounding	1.5	Evaluation includes quantitative metrics and qualitative analysis; identifies strengths and weaknesses

Criteria	Points	Description
Explanation of chunking/indexing design decisions	1	Clear justification for chunk size, overlap, embedding model, and indexing strategy
Pipeline diagram clarity and reproducibility	0.5	Diagram accurately represents system architecture; environment setup, model-serving instructions, and reproducibility details are complete and functional

Requirements & Constraints

- **Python only**
- Must run **locally or in Google Colab**
- Vector database must be **open-source** (FAISS, Chroma, Weaviate, or equivalent)
- Final evaluation must use a **real open-weight instruct LLM in the 7B-14B class**
- Model inference must be **local or self-hosted**; proprietary hosted APIs are not allowed without approval
- Document your design decisions clearly
- Mocks may be used for debugging, but **not** for final submitted results

---

Part 2: Multi-Tool Agent with Retrieval Integration (5 points)

Requirements

You will implement a lightweight **agent controller** that decides when to retrieve information versus when to perform another action (summarization, extraction, reasoning, etc.). Your agent must solve real multi-step tasks using retrieval as one of multiple available tools. The final submitted agent must use a ** real open-weight 7B-14B instruct LLM** in the control loop, answer generation step, or both.

Deliverables

1. **Agent controller implementation** (custom logic or framework-based)
    - Must demonstrate intelligent tool selection
    - Should expose decision-making logic
    - Must integrate a real open-weight 7B-14B instruct LLM for final evaluated runs
2. **At least two tools**, including:
    - **Retriever** (from Part 1 or standalone)
    - **One additional tool:** summarizer, classifier, reasoning module, extraction tool, or similar

3. **10 multi-step evaluation tasks** with agent traces showing:
    - **Decision points:** When and why the agent chose specific tools
    - **Tool selection logic:** Observable reasoning for tool choices
    - **Retrieval results:** What information was retrieved and how it was used
    - **Final outputs:** Complete task solutions with evidence trails
4. **Short report** (1-2 pages) explaining:
    - **Policy for tool selection:** How your agent decides which tool to use
    - **Retrieval integration:** How retrieval coordinates with other agent capabilities
    - **Performance analysis:** Evaluation of agent performance on the 10 tasks
    - **Failure analysis:** Cases where the agent struggled or failed
    - **Model analysis:** How the chosen open-weight model affected quality, latency, and failure modes
5. **Environment file** (if different from Part 1)

Rubric (5 pts)

Criteria	Points	Description
Working agent that selects tools in multi-step workflow	2	Agent successfully coordinates multiple tools with a real open-weight 7B-14B instruct LLM in the workflow; demonstrates autonomous decision-making
Correct integration of retrieval as decision-triggered tool	1.5	Retrieval is properly integrated; agent uses it appropriately based on task requirements
Quality of reasoning traces + transparency of decisions	1	Agent traces clearly show decision logic; intermediate steps are observable and interpretable
Evaluation task diversity and documentation	0.5	Tasks cover diverse scenarios; documentation clearly explains agent behavior

Requirements & Constraints

- Final evaluation must use a **real open-weight instruct LLM in the 7B-14B class**
- **No external proprietary APIs allowed** without explicit instructor approval
- Local or self-hosted serving is required (local machine or cloud VM/notebook you control)

- Agent must **expose intermediate reasoning steps** (tool decisions and states)
- **Retrieval component should be reusable** from Part 1 where possible
- Document your agent's decision-making policy clearly
- Purely mock, rule-only, or extractive-only substitutes are acceptable for debugging but **not** for final full-credit submission

---

Combined Deliverables Summary

Submit the following files:

1. **RAG pipeline implementation** (code + notebook)
2. **Agent controller implementation** (code + notebook)
3. **Comprehensive evaluation report** covering:
    - RAG retrieval accuracy and grounding (Part 1)
    - Agent task performance and tool selection (Part 2)
    - Latency analysis for both components
4. **Pipeline diagram** (Part 1)
5. **Agent trace examples** (Part 2, minimum 10 tasks)
6. **Environment files** and complete documentation
7. **README** with:
    - Setup instructions
    - Usage examples
    - Architecture overview
    - Known limitations
    - Model name, size class, serving stack, and hardware/runtime details

---

What Success Looks Like

A high-quality submission will:

- Implement a fully functional RAG pipeline with clear design rationale
- Build an agent that demonstrates intelligent, observable tool coordination
- Run a real open-weight 7B-14B instruct model and document the serving setup clearly
- Provide rigorous evaluation with both quantitative metrics and qualitative analysis
- Include comprehensive documentation enabling easy reproduction
- Show thoughtful reflection on system performance and limitations

---

Getting Started

Follow these steps to complete this milestone:

Part 1: RAG Pipeline

1. **Choose a vector database** (FAISS for simplicity, Chroma for features)
2. **Prepare a small document corpus** (5-10 documents for initial testing)
3. **Implement document chunking** with your chosen strategy (fixed size, semantic, etc.)
4. **Generate embeddings** using a sentence transformer or similar model
5. **Build and test retrieval** with simple queries before adding generation
6. **Deploy an open-weight 7B-14B instruct LLM** using Ollama, vLLM, or Transformers
7. **Integrate the LLM** for grounded answer generation
8. **Record model latency and failure cases** during evaluation
9. **Create 10 test queries** and evaluate retrieval accuracy

Part 2: Agent Controller

1. **Design your tool interface** defining how tools are called and return results
2. **Implement the retriever as a tool** that the agent can invoke
3. **Add a second tool** (summarizer, classifier, or reasoning module)
4. **Integrate the same or another open-weight 7B-14B instruct model** into the agent workflow
5. **Build agent decision logic** that selects tools based on task requirements
6. **Create 10 multi-step tasks** that require tool coordination
7. **Log all agent decisions** to create observable traces

Tips for Success

For Part 1 (RAG Pipeline):

- **Start simple:** Get basic retrieval working before optimizing
- **Test chunking strategies:** Experiment with different chunk sizes and overlap
- **Measure everything:** Track latency at each pipeline stage
- **Evaluate thoroughly:** Use diverse queries that test different aspects of retrieval
- **Run a real model early:** Validate your 7B-14B model deployment before building the full pipeline
- **Document decisions:** Explain why you chose specific embeddings, chunk sizes, etc.

For Part 2 (Agent Controller):

- **Make decisions observable:** Log every tool selection and reasoning step
- **Start with simple tasks:** Build complexity gradually
- **Test edge cases:** What happens when retrieval fails? When tools conflict?
- **Track state carefully:** Maintain clear context across multi-step execution
- **Measure model behavior:** Note when the chosen model follows tool instructions well or poorly
- **Analyze failures:** Understanding when and why the agent fails is as important as successes

Challenge Extensions (Optional)

Want to go further? Consider these optional enhancements:

Part 1 Extensions:

- **Reranking:** Implement a reranker and compare retrieval performance before vs. after reranking
- **Hybrid search:** Combine dense and sparse retrieval methods
- **Query expansion:** Implement automatic query reformulation or expansion

Part 2 Extensions:

- **Recovery mechanisms:** Allow the agent to recover from bad retrieval (e.g., rerun with modified query)
- **Confidence scoring:** Add confidence estimates for tool selections and outputs
- **Parallel execution:** Enable concurrent tool use when tasks are independent

Combined Extension:

- **Unified system:** Integrate both components into a single system with shared vector store and coordinated execution
- **Monitoring dashboard:** Add real-time monitoring of retrieval quality and agent decisions

Submission Requirements

Create a new GitHub repository named `ids568-milestone6-[your_netid]` (e.g., `ids568-milestone6-jsmith`).

Your repository should contain all deliverables **at the root level**:

```
ids568-milestone6-[netid]/
├── rag_pipeline.ipynb (or .py)      # RAG implementation
├── agent_controller.py (or .ipynb)  # Agent implementation
├── rag_evaluation_report.md         # Part 1 evaluation
├── rag_pipeline_diagram.md          # Pipeline architecture
├── agent_report.md                  # Part 2 analysis
├── agent_traces/                    # Multi-step task traces
├── requirements.txt                 # Pinned dependencies
└── README.md                        # Setup and usage
```

Before submitting:

1. Test setup on a clean environment
2. Verify all 10 RAG queries and 10 agent tasks are documented
3. Verify your README documents the exact open-weight model and serving setup used for final runs
4. Tag your final submission: `git tag submission && git push --tags`

Submit via Course Submission Site: Your repository URL

After grades are finalized, you may delete this repository to reduce clutter.

> **Alternative submission:** If you have constraints preventing a new repository, contact the instructor to arrange branch-based submission.

Submission Deadline: [To be announced]

Setup Guidance

Vector Database Setup

```
# Install FAISS (CPU version)
pip install faiss-cpu

# OR install Chroma
pip install chromadb
```

Embedding Model Setup

```
# Install sentence transformers
pip install sentence-transformers

# Test embedding generation
python -c "from sentence_transformers import SentenceTransformer; model =
    SentenceTransformer('all-MiniLM-L6-v2'); print('Model loaded
    successfully')"
```

LLM Setup (Required for Final Submission)

For final evaluation, you must run a **real open-weight instruct model in the 7B-14B class**. The simplest supported path is ** Ollama**. `vLLM` is a strong option if you are running on a cloud GPU and want a more production-like serving stack.

Option	Recommended for	Notes
Ollama	Fastest setup on laptop or VM	Good default for most students
vLLM	Cloud GPU or higher-throughput serving	Better serving ergonomics, more setup
Transformers	Custom experiments	Most flexible, but easiest to misconfigure

Suggested models:

Model	Size Class	Notes
`mistralai/Mistral-7B-Instruct-v0.3`	7B	Strong default for RAG and instruction following
`meta-llama/Llama-3.1-8B-Instruct`	8B	Good general-purpose option
`Qwen/Qwen2.5-7B-Instruct`	7B	Good balance of quality and efficiency
`Qwen/Qwen2.5-14B-Instruct`	14B	Better quality if your credits/hardware support it

```
# Option 1: Ollama (recommended)
# Install Ollama, then pull a supported model
ollama pull mistral:7b-instruct

# Option 2: Hugging Face Transformers
pip install transformers torch accelerate

# Option 3: vLLM for cloud GPU serving
pip install vllm
```

Resources

- FAISS Documentation
- Chroma Documentation
- Sentence Transformers
- LangChain RAG Tutorial
- LlamaIndex Getting Started
- DSPy Documentation
- Ollama
- vLLM

Total Points: 10

This milestone directly aligns with **CG3.LO3** and ** CG3.LO4**, preparing you for the final project's monitoring and governance requirements by establishing the foundational RAG and agentic systems you'll need to observe, evaluate, and maintain in production.

Evaluation Criteria

Your work will be evaluated on:

- **Technical correctness** of RAG and agent implementations
- **Correct integration of a real open-weight 7B-14B instruct model**
- **Rigor of evaluation methodology** and metrics
- **Clarity of documentation** and architectural explanations
- **Depth of analysis** in understanding system behavior and limitations
- **Reproducibility** of all components and experiments